

AI ASSISTED CODING ASS – 17.1

2303A510H2

B-27

Task 1 – Customer Data Cleaning

Task: Use AI to generate a Python script for cleaning a customer dataset.

Instructions:

- Handle missing values in columns (age, income, city).
- Remove duplicate records.
- Standardize text values (e.g., city names in lowercase).
- Encode categorical variables (gender, city).

Prompt: Generate a Python script using Pandas to clean shopping_trends.csv. Handle missing values (age → median, income → mean, city → mode), remove duplicates, convert city to lowercase and strip spaces, encode gender (label encoding) and city (one-hot encoding), and display the cleaned dataset with info.

CODE:

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder

df = pd.read_csv('/content/shopping_trends.csv')
print('Initial DataFrame Info:')
display(df.info())
print('\nFirst 5 rows of the original DataFrame:')
display(df.head())

Initial DataFrame Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3900 entries, 0 to 3899
Data columns (total 19 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Customer ID                          3900 non-null   int64
1   Age                                  3900 non-null   int64
2   Gender                              3900 non-null   object
3   Item Purchased                       3900 non-null   object
4   Category                            3900 non-null   object
5   Purchase Amount (USD)                3900 non-null   int64
6   Location                             3900 non-null   object
7   Size                                 3900 non-null   object
8   Color                                3900 non-null   object
9   Season                               3900 non-null   object
10  Review Rating                        3900 non-null   float64
11  Subscription Status                  3900 non-null   object
12  Payment Method                      3900 non-null   object
13  Shipping Type                       3900 non-null   object
14  Discount Applied                    3900 non-null   object
15  Promo Code Used                     3900 non-null   object
16  Previous Purchases                   3900 non-null   int64
17  Preferred Payment Method             3900 non-null   object
18  Frequency of Purchases                3900 non-null   object
dtypes: float64(1), int64(4), object(14)
memory usage: 579.0+ KB
None
```

Handle Missing Values

I will fill missing values in 'Age' with the median, 'Income' with the mean, and 'City' with the mode.

```
# Fill missing 'Age' with median
df['Age'].fillna(df['Age'].median(), inplace=True)

# Fill missing 'Income' with mean
df['Purchase Amount (USD)'].fillna(df['Purchase Amount (USD)'].mean(), inplace=True)

# Fill missing 'City' with mode
mode_city = df['Location'].mode()[0]
df['Location'].fillna(mode_city, inplace=True)

print('Missing values after handling:')
display(df.isnull().sum())
```

```
df['Location'].fillna(mode_city, inplace=True)
```

	0
Customer ID	0
Age	0
Gender	0
Item Purchased	0
Category	0
Purchase Amount (USD)	0
Location	0
Size	0
Color	0
Season	0
Review Rating	0
Subscription Status	0
Payment Method	0
Shipping Type	0
Discount Applied	0
Promo Code Used	0
Previous Purchases	0
Preferred Payment Method	0
Frequency of Purchases	0

dtype: int64

Remove Duplicate Rows

I will remove any duplicate rows present in the dataset.

```
initial_rows = df.shape[0]
df.drop_duplicates(inplace=True)
rows_after_deduplication = df.shape[0]
print(f'Number of duplicate rows removed: {initial_rows - rows_after_deduplication}')
```

Number of duplicate rows removed: 0

Clean 'City' Column

I will convert the 'Location' (city) column to lowercase and strip leading/trailing spaces.

```
df['Location'] = df['Location'].str.lower().str.strip()
print('\nUnique cities after cleaning:')
display(df['Location'].unique())
```

Unique cities after cleaning:

```
array(['kentucky', 'maine', 'massachusetts', 'rhode island', 'oregon',
       'wyoming', 'montana', 'louisiana', 'west virginia', 'missouri',
       'arkansas', 'hawaii', 'delaware', 'new hampshire', 'new york',
       'alabama', 'mississippi', 'north carolina', 'california',
       'oklahoma', 'florida', 'texas', 'nevada', 'kansas', 'colorado',
       'north dakota', 'illinois', 'indiana', 'arizona', 'alaska',
       'tennessee', 'ohio', 'new jersey', 'maryland', 'vermont',
       'new mexico', 'south carolina', 'idaho', 'pennsylvania',
       'connecticut', 'utah', 'virginia', 'georgia', 'nebraska', 'iowa',
       'south dakota', 'minnesota', 'washington', 'wisconsin', 'michigan'],
      dtype=object)
```

Display Cleaned Dataset Information

Finally, here is the information for the cleaned dataset and its first 5 rows.

```
print('\nCleaned DataFrame Info:')
display(df.info())
print('\nFirst 5 rows of the cleaned DataFrame:')
display(df.head())
```

```
Cleaned DataFrame Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3900 entries, 0 to 3899
Data columns (total 69 columns):
 #   Column                                Non-Null Count  Dtype
---  -
 0   Customer ID                          3900 non-null   int64
 1   Age                                  3900 non-null   int64
 2   Gender                               3900 non-null   object
 3   Item Purchased                       3900 non-null   object
 4   Category                             3900 non-null   object
 5   Purchase Amount (USD)                3900 non-null   int64
 6   Size                                 3900 non-null   object
 7   Color                                3900 non-null   object
 8   Season                               3900 non-null   object
 9   Review Rating                        3900 non-null   float64
10   Subscription Status                  3900 non-null   object
11   Payment Method                       3900 non-null   object
12   Shipping Type                        3900 non-null   object
13   Discount Applied                     3900 non-null   object
14   Promo Code Used                      3900 non-null   object
15   Previous Purchases                   3900 non-null   int64
16   Preferred Payment Method             3900 non-null   object
17   Frequency of Purchases               3900 non-null   object
18   Gender_encoded                       3900 non-null   int64
```

First 5 rows of the cleaned DataFrame:

	Customer ID	Age	Gender	Item Purchased	Category	Purchase Amount (USD)	Size	Color	Season	Review Rating	...	Location_south dakota	Location_tennessee	Location_texas	Location_utah	Location_vermont	Location_virginia
0	1	55	Male	Blouse	Clothing	53	L	Gray	Winter	3.1	...	False	False	False	False	False	False
1	2	19	Male	Sweater	Clothing	64	L	Maroon	Winter	3.1	...	False	False	False	False	False	False
2	3	50	Male	Jeans	Clothing	73	S	Maroon	Spring	3.1	...	False	False	False	False	False	False
3	4	21	Male	Sandals	Footwear	90	M	Maroon	Spring	3.5	...	False	False	False	False	False	False
4	5	45	Male	Blouse	Clothing	49	M	Turquoise	Spring	2.7	...	False	False	False	False	False	False

5 rows × 69 columns

Task 2 – Sales Data Preprocessing

Task: Preprocess sales transaction data using AI assistance.

Instructions:

- Convert date columns into datetime format.
- Create new features (month, year, day of week).
- Normalize sales amount column.
- Detect and handle outliers in dataset.

Prompt: Generate a Python script using Pandas to preprocess *sales_data.csv*. Convert date columns to datetime, create new features (month, year, day of week), normalize the sales amount column, detect and handle outliers, and display the final dataset.

CODE:

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler

# Load the dataset
df = pd.read_csv('/content/sales_data.csv')

print('Initial DataFrame Info:')
display(df.info())
print('\nFirst 5 rows of the original DataFrame:')
display(df.head())

Initial DataFrame Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 14 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Product_ID            1000 non-null   int64
1   Sale_Date              1000 non-null   object
2   Sales_Rep              1000 non-null   object
3   Region                 1000 non-null   object
4   Sales_Amount           1000 non-null   float64
5   Quantity_Sold          1000 non-null   int64
6   Product_Category       1000 non-null   object
7   Unit_Cost              1000 non-null   float64
8   Unit_Price             1000 non-null   float64
9   Customer_Type          1000 non-null   object
10  Discount               1000 non-null   float64
11  Payment_Method         1000 non-null   object
12  Sales_Channel          1000 non-null   object
13  Region_and_Sales_Rep   1000 non-null   object
dtypes: float64(4), int64(2), object(8)
memory usage: 109.5+ KB
None
```

Convert Date Columns and Create New Features

Next, I will convert the 'Purchase Date' column to datetime objects and extract 'Month', 'Year', and 'Day of Week' as new features.

```
# Convert 'Sale_Date' to datetime
df['Sale_Date'] = pd.to_datetime(df['Sale_Date'])

# Create new features: Month, Year, Day of Week
df['Sale Month'] = df['Sale_Date'].dt.month
df['Sale Year'] = df['Sale_Date'].dt.year
df['Sale DayOfWeek'] = df['Sale_Date'].dt.dayofweek

print('\nDataFrame after adding date features:')
display(df[['Sale_Date', 'Sale Month', 'Sale Year', 'Sale DayOfWeek']].head())
```

DataFrame after adding date features:

	Sale_Date	Sale Month	Sale Year	Sale DayOfWeek
0	2023-02-03	2	2023	4
1	2023-04-21	4	2023	4
2	2023-09-21	9	2023	3
3	2023-08-24	8	2023	3
4	2023-03-24	3	2023	4

Normalize Sales Amount Column

I will normalize the 'Sales_Amount' column using Min-Max scaling to bring its values into a specific range, typically between 0 and 1.

```
# Initialize MinMaxScaler
scaler = MinMaxScaler()

# Normalize 'Sales_Amount'
df['Normalized_Sales_Amount'] = scaler.fit_transform(df[['Sales_Amount']])

print('\nDataFrame after normalizing Sales Amount:')
display(df[['Sales_Amount', 'Normalized_Sales_Amount']].head())
```

DataFrame after normalizing Sales Amount:

	Sales_Amount	Normalized_Sales_Amount
0	5053.97	0.500950
1	4384.02	0.433202
2	4631.23	0.458201
3	2167.94	0.209105
4	3750.20	0.369108

Detect and Handle Outliers

I will use the Interquartile Range (IQR) method to detect and handle outliers in the 'Sales_Amount' column. Outliers will be capped to the upper and lower bounds.

```
# Calculate IQR for 'Sales_Amount'
Q1 = df['Sales_Amount'].quantile(0.25)
Q3 = df['Sales_Amount'].quantile(0.75)
IQR = Q3 - Q1

# Define outlier bounds
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

# Cap outliers
df['Sales_Amount'] = df['Sales_Amount'].clip(lower=lower_bound, upper=upper_bound)

print(f'\nOutliers handled for Sales_Amount. Lower bound: {lower_bound:.2f}, Upper bound: {upper_bound:.2f}')
print('Summary statistics after handling outliers:')
display(df['Sales_Amount'].describe())
```

Outliers handled for Sales_Amount. Lower bound: -4885.42, Upper bound: 14943.17
Summary statistics after handling outliers:

Sales_Amount	
count	1000.000000
mean	5019.265230
std	2846.790126
min	100.120000
25%	2550.297500
50%	5019.300000
75%	7507.445000
max	9989.040000

dtype: float64

Display Final Dataset

Finally, here is the information for the processed dataset and its first 5 rows.

```
print('\nProcessed DataFrame Info:')
display(df.info())
print('\nFirst 5 rows of the processed DataFrame:')
display(df.head())
```

```
Processed DataFrame Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 18 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Product_ID                           1000 non-null   int64
1   Sale_Date                             1000 non-null   datetime64[ns]
2   Sales_Rep                             1000 non-null   object
3   Region                                1000 non-null   object
4   Sales_Amount                          1000 non-null   float64
5   Quantity_Sold                         1000 non-null   int64
6   Product_Category                      1000 non-null   object
7   Unit_Cost                             1000 non-null   float64
8   Unit_Price                            1000 non-null   float64
9   Customer_Type                         1000 non-null   object
10  Discount                              1000 non-null   float64
11  Payment_Method                        1000 non-null   object
12  Sales_Channel                         1000 non-null   object
13  Region_and_Sales_Rep                  1000 non-null   object
14  Sale_Month                            1000 non-null   int32
15  Sale_Year                             1000 non-null   int32
16  Sale_DayOfWeek                        1000 non-null   int32
17  Normalized_Sales_Amount               1000 non-null   float64
dtypes: datetime64[ns](1), float64(5), int32(3), int64(2), object(7)
memory usage: 129.0+ KB
None
```

Task 3 – Healthcare Data Preparation

Task: Clean and preprocess a healthcare dataset using AI scripts.

Instructions:

- Handle missing values in `blood_pressure`, `cholesterol`.
- Encode categorical features
- Scale numerical features (min-max or standard scaling).
- Split dataset into training and testing sets.

Prompt: Generate a Python script using Pandas and Scikit-learn to preprocess a healthcare dataset. Handle missing values in *blood_pressure* and *cholesterol*, encode categorical features, scale numerical features (min-max or standard scaling), split the dataset into training and testing sets, and display the processed output.

CODE:

Data Preprocessing for Healthcare Dataset

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler, OneHotEncoder
from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer
```

1. Create a Synthetic Healthcare Dataset

Since no specific dataset was provided, I'll create a synthetic healthcare dataset for demonstration purposes. This dataset will include features like age, gender, blood pressure, cholesterol, and a target variable (e.g., 'diagnosis').

```
# Generate synthetic data
np.random.seed(42)

data_size = 1000

data = {
    'age': np.random.randint(20, 80, data_size),
    'gender': np.random.choice(['Male', 'Female'], data_size),
    'blood_pressure': np.random.normal(120, 15, data_size),
    'cholesterol': np.random.normal(200, 40, data_size),
    'weight': np.random.normal(70, 10, data_size),
    'smoker': np.random.choice([0, 1], data_size, p=[0.7, 0.3]),
    'diagnosis': np.random.choice(['Healthy', 'Ill'], data_size, p=[0.6, 0.4])
}

df = pd.DataFrame(data)

# Introduce some missing values for demonstration
for col in ['blood_pressure', 'cholesterol']:
    missing_indices = np.random.choice(df.index, int(data_size * 0.05), replace=False)
    df.loc[missing_indices, col] = np.nan

print("Original DataFrame Info:")
display(df.info())
print("\nFirst 5 rows of the original DataFrame:")
display(df.head())
```

```
Original DataFrame Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 7 columns):
#   Column      Non-Null Count  Dtype
---  -
0   age         1000 non-null   int64
1   gender      1000 non-null   object
2   blood_pressure  950 non-null   float64
3   cholesterol  950 non-null   float64
4   weight      1000 non-null   float64
5   smoker      1000 non-null   int64
6   diagnosis   1000 non-null   object
dtypes: float64(3), int64(2), object(2)
memory usage: 54.8+ KB
None
```

First 5 rows of the original DataFrame:

	age	gender	blood_pressure	cholesterol	weight	smoker	diagnosis
0	58	Female	135.152055	243.054880	74.338809	1	Healthy
1	71	Female	117.056082	223.694077	43.732137	1	III
2	48	Male	103.588438	173.189999	70.429322	0	Healthy
3	34	Female	84.425765	210.696279	83.446761	0	III
4	62	Male	NaN	245.281399	70.943086	0	III

2. Handle Missing Values

Missing values in 'blood_pressure' and 'cholesterol' will be imputed using the mean of their respective columns. For other columns, we will drop rows with missing values if any remain (though for this synthetic data, there shouldn't be).

```
# Initialize Imputer for numerical columns
imputer_numerical = SimpleImputer(strategy='mean')

# Impute missing values in 'blood_pressure' and 'cholesterol'
df[['blood_pressure', 'cholesterol']] = imputer_numerical.fit_transform(df[['blood_pressure', 'cholesterol']])

print("\nDataFrame Info after handling missing values:")
display(df.info())
print("\nFirst 5 rows after imputation:")
display(df.head())
```

```
DataFrame Info after handling missing values:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 7 columns):
#   Column      Non-Null Count  Dtype
---  -
0   age         1000 non-null   int64
1   gender      1000 non-null   object
2   blood_pressure  1000 non-null   float64
3   cholesterol  1000 non-null   float64
4   weight      1000 non-null   float64
5   smoker      1000 non-null   int64
6   diagnosis   1000 non-null   object
dtypes: float64(3), int64(2), object(2)
memory usage: 54.8+ KB
None
```

First 5 rows after imputation:

	age	gender	blood_pressure	cholesterol	weight	smoker	diagnosis
0	58	Female	135.152055	243.054880	74.338809	1	Healthy
1	71	Female	117.056082	223.694077	43.732137	1	III
2	48	Male	103.588438	173.189999	70.429322	0	Healthy
3	34	Female	84.425765	210.696279	83.446761	0	III
4	62	Male	120.556126	245.281399	70.943086	0	III

3. Encode Categorical Features

Categorical features like 'gender' and 'diagnosis' will be one-hot encoded to convert them into a numerical format suitable for machine learning models.

```
# Identify categorical features
categorical_features = ['gender', 'diagnosis']

# Apply One-Hot Encoding
encoder = OneHotEncoder(handle_unknown='ignore', sparse_output=False)
encoded_features = encoder.fit_transform(df[categorical_features])

# Create a DataFrame from encoded features
encoded_df = pd.DataFrame(encoded_features, columns=encoder.get_feature_names_out(categorical_features))

# Drop original categorical columns and concatenate encoded ones
df = df.drop(columns=categorical_features)
df = pd.concat([df, encoded_df], axis=1)

print("\nDataFrame Info after one-hot encoding:")
display(df.info())
print("\nFirst 5 rows after one-hot encoding:")
display(df.head())
```

```
DataFrame Info after one-hot encoding:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 9 columns):
#   Column              Non-Null Count  Dtype
---  -
0   age                  1000 non-null   int64
1   blood_pressure       1000 non-null   float64
2   cholesterol          1000 non-null   float64
3   weight              1000 non-null   float64
4   smoker              1000 non-null   int64
5   gender_Female        1000 non-null   float64
6   gender_Male          1000 non-null   float64
7   diagnosis_Healthy    1000 non-null   float64
8   diagnosis_Ill        1000 non-null   float64
dtypes: float64(7), int64(2)
memory usage: 70.4 KB
None

First 5 rows after one-hot encoding:
```

	age	blood_pressure	cholesterol	weight	smoker	gender_Female	gender_Male	diagnosis_Healthy	diagnosis_Ill
0	58	135.152055	243.054880	74.338809	1	1.0	0.0	1.0	0.0
1	71	117.056082	223.694077	43.732137	1	1.0	0.0	0.0	1.0
2	48	103.588438	173.189999	70.429322	0	0.0	1.0	1.0	0.0
3	34	84.425765	210.696279	83.446761	0	1.0	0.0	0.0	1.0
4	62	120.556126	245.281399	70.943086	0	0.0	1.0	0.0	1.0

4. Scale Numerical Features

Numerical features ('age', 'blood_pressure', 'cholesterol', 'weight') will be scaled using MinMaxScaler to bring their values into a range between 0 and 1.

```
# Identify numerical features for scaling (excluding binary and encoded categorical features)
numerical_features = ['age', 'blood_pressure', 'cholesterol', 'weight']

# Apply Min-Max Scaling
scaler = MinMaxScaler()
df[numerical_features] = scaler.fit_transform(df[numerical_features])

print("\nDataFrame Info after scaling numerical features:")
display(df.info())
print("\nFirst 5 rows after scaling:")
display(df.head())
```

5. Split the Dataset into Training and Testing Sets

The dataset will be split into features (X) and target (y), and then further divided into training and testing sets to prepare for model training and evaluation.

```

# Assuming 'diagnosis_Ill' or 'diagnosis_Healthy' is the target variable
# Let's use 'diagnosis_Ill' as the target for this example

X = df.drop(columns=['diagnosis_Healthy', 'diagnosis_Ill']) # Features
y = df['diagnosis_Ill'] # Target

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

print(f"\nShape of X_train: {X_train.shape}")
print(f"Shape of X_test: {X_test.shape}")
print(f"Shape of y_train: {y_train.shape}")
print(f"Shape of y_test: {y_test.shape}")

print("\nFirst 5 rows of X_train:")
display(X_train.head())
print("\nFirst 5 rows of y_train:")
display(y_train.head())

```

```

Shape of X_train: (800, 7)
Shape of X_test: (200, 7)
Shape of y_train: (800,)
Shape of y_test: (200,)

```

First 5 rows of X_train:

	age	blood_pressure	cholesterol	weight	smoker	gender_Female	gender_Male
218	0.966102	0.745720	0.592892	0.651212	0	1.0	0.0
338	1.000000	0.527877	0.672702	0.595153	0	0.0	1.0
803	0.966102	0.612119	0.682044	0.674437	0	0.0	1.0
777	0.220339	0.598739	0.409061	0.663007	0	1.0	0.0
82	0.016949	0.470703	0.494158	0.374251	0	0.0	1.0

First 5 rows of y_train:

	diagnosis_Ill
218	0.0
338	0.0
803	1.0
777	0.0
82	1.0

dtype: float64

6. Display Processed Output Summary

Here is a summary of the final processed training data.

```

print("\nProcessed Training Data Info:")
display(X_train.info())
print("\nFirst 5 rows of the final processed training data:")
display(X_train.head())

```

```
Processed Training Data Info:
<class 'pandas.core.frame.DataFrame'>
Index: 800 entries, 218 to 5
Data columns (total 7 columns):
#   Column          Non-Null Count  Dtype
---  ---
0    age             800 non-null    float64
1   blood_pressure   800 non-null    float64
2   cholesterol      800 non-null    float64
3    weight          800 non-null    float64
4    smoker          800 non-null    int64
5   gender_Female    800 non-null    float64
6   gender_Male      800 non-null    float64
dtypes: float64(6), int64(1)
memory usage: 50.0 KB
None

First 5 rows of the final processed training data:
```

	age	blood_pressure	cholesterol	weight	smoker	gender_Female	gender_Male
218	0.966102	0.745720	0.592892	0.651212	0	1.0	0.0
338	1.000000	0.527877	0.672702	0.595153	0	0.0	1.0
803	0.966102	0.612119	0.682044	0.674437	0	0.0	1.0
777	0.220339	0.598739	0.409061	0.663007	0	1.0	0.0
82	0.016949	0.470703	0.494158	0.374251	0	0.0	1.0

Task 4 – Employee Salary Data Preprocessing

Task: Clean and preprocess an employee salary dataset using AI scripts.

Instructions:

- Handle missing values
- Detect and remove salary outliers.
- Standardize department names (lowercase).
- Apply label encoding to job location, department.

Prompt: Generate a Python script using Pandas to preprocess an employee salary dataset. Handle missing values, detect and remove salary outliers, convert department names to lowercase, apply label encoding to job location and department, and display the cleaned dataset.

CODE:

Data Preprocessing Steps

First, let's load the dataset and display its initial information.

```
import pandas as pd

# Load the dataset
try:
    df_salary = pd.read_csv('/content/Employee_Salary_Dataset.csv')
    print("Dataset loaded successfully.")
except FileNotFoundError:
    print("Error: 'Employee_Salary_Dataset.csv' not found. Please ensure the file is uploaded to '/content/'.")
    df_salary = pd.DataFrame() # Create an empty DataFrame to avoid errors later

# Display the first 5 rows and information about the dataset
if not df_salary.empty:
    display(df_salary.head())
    print("\nDataset Information:")
    df_salary.info()
```

Dataset loaded successfully.

	ID	Experience_Years	Age	Gender	Salary
0	1	5	28	Female	250000
1	2	1	21	Male	50000
2	3	3	23	Female	170000
3	4	2	22	Male	25000
4	5	1	17	Male	10000

Dataset Information:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 35 entries, 0 to 34

Data columns (total 5 columns):

#	Column	Non-Null Count	Dtype
0	ID	35 non-null	int64
1	Experience_Years	35 non-null	int64
2	Age	35 non-null	int64
3	Gender	35 non-null	object
4	Salary	35 non-null	int64

dtypes: int64(4), object(1)

memory usage: 1.5+ KB

Handling Missing Values

We'll check for missing values and fill numerical columns with the median and categorical columns with the mode. If a column is identified as potentially having salary outliers, it's best to handle missing values before outlier detection to ensure accurate calculations for quartiles.

```
if not df_salary.empty:
    print("\nMissing values before handling:")
    print(df_salary.isnull().sum())

    # Identify numerical and categorical columns
    numerical_cols = df_salary.select_dtypes(include=['number']).columns
    categorical_cols = df_salary.select_dtypes(include=['object', 'category']).columns

    # Fill missing values for numerical columns with the median
    for col in numerical_cols:
        if df_salary[col].isnull().any():
            median_val = df_salary[col].median()
            df_salary[col].fillna(median_val, inplace=True)
            print(f"Filled missing values in '{col}' with median: {median_val}")

    # Fill missing values for categorical columns with the mode
    for col in categorical_cols:
        if df_salary[col].isnull().any():
            mode_val = df_salary[col].mode()[0] # mode() can return multiple values, take the first
            df_salary[col].fillna(mode_val, inplace=True)
            print(f"Filled missing values in '{col}' with mode: {mode_val}")

    print("\nMissing values after handling:")
    print(df_salary.isnull().sum())
```

```
Missing values before handling:
ID          0
Experience_Years  0
Age         0
Gender      0
Salary      0
dtype: int64

Missing values after handling:
ID          0
Experience_Years  0
Age         0
Gender      0
Salary      0
dtype: int64
```

Detecting and Removing Salary Outliers

We will use the Interquartile Range (IQR) method to detect and remove outliers from the 'salary' column. This method defines outliers as data points that fall below $Q1 - 1.5 * IQR$ or above $Q3 + 1.5 * IQR$.

```
if not df_salary.empty and 'salary' in df_salary.columns:
    # Calculate Q1, Q3, and IQR for the 'salary' column
    Q1 = df_salary['salary'].quantile(0.25)
    Q3 = df_salary['salary'].quantile(0.75)
    IQR = Q3 - Q1

    # Define outlier bounds
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    # Identify outliers
    outliers = df_salary[(df_salary['salary'] < lower_bound) | (df_salary['salary'] > upper_bound)]

    print(f"Number of salary outliers detected: {len(outliers)}")
    if not outliers.empty:
        print("Examples of salary outliers:")
        display(outliers.head())

    # Remove outliers
    df_salary_cleaned = df_salary[~((df_salary['salary'] < lower_bound) | (df_salary['salary'] > upper_bound))]
    print(f"\nDataset shape before outlier removal: {df_salary.shape}")
    print(f"Dataset shape after outlier removal: {df_salary_cleaned.shape}")
    df_salary = df_salary_cleaned # Update the DataFrame
else:
    print("Skipping outlier detection: 'salary' column not found or DataFrame is empty.")

Skipping outlier detection: 'salary' column not found or DataFrame is empty.
```

Convert Department Names to Lowercase

Standardizing categorical data, such as department names, is crucial for consistency. We'll convert all department names to lowercase.

```
if not df_salary.empty and 'department' in df_salary.columns:
    print("\nDepartment names before conversion:")
    print(df_salary['department'].unique())

    df_salary['department'] = df_salary['department'].str.lower()

    print("\nDepartment names after conversion to lowercase:")
    print(df_salary['department'].unique())
else:
    print("Skipping department name conversion: 'department' column not found or DataFrame is empty.")

Skipping department name conversion: 'department' column not found or DataFrame is empty.
```


Apply Label Encoding to Job Location and Department

Label Encoding converts categorical labels into numerical format, which is often required for machine learning models. We'll apply this to the 'job_location' and 'department' columns.

```
from sklearn.preprocessing import LabelEncoder

if not df_salary.empty:
    le = LabelEncoder()

    # Apply Label Encoding to 'job_location'
    if 'job_location' in df_salary.columns:
        df_salary['job_location_encoded'] = le.fit_transform(df_salary['job_location'])
        print("\nOriginal Job Location vs. Encoded:")
        display(df_salary[['job_location', 'job_location_encoded']].head())
    else:
        print("Skipping 'job_location' encoding: Column not found.")

    # Apply Label Encoding to 'department'
    if 'department' in df_salary.columns:
        df_salary['department_encoded'] = le.fit_transform(df_salary['department'])
        print("\nOriginal Department vs. Encoded:")
        display(df_salary[['department', 'department_encoded']].head())
    else:
        print("Skipping 'department' encoding: Column not found.")
else:
    print("Skipping label encoding: DataFrame is empty.")

Skipping 'job_location' encoding: Column not found.
Skipping 'department' encoding: Column not found.
```

Display Cleaned Dataset

Finally, let's look at the first few rows and the information of the fully preprocessed dataset.

```
if not df_salary.empty:
    print("\nCleaned Dataset Head:")
    display(df_salary.head())
    print("\nCleaned Dataset Information:")
    df_salary.info()
else:
    print("No data to display as the DataFrame is empty after preprocessing.")
```

Cleaned Dataset Head:

	ID	Experience_Years	Age	Gender	Salary
0	1	5	28	Female	250000
1	2	1	21	Male	50000
2	3	3	23	Female	170000
3	4	2	22	Male	25000
4	5	1	17	Male	10000

Cleaned Dataset Information:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 35 entries, 0 to 34
Data columns (total 5 columns):
#   Column          Non-Null Count  Dtype
---  -
0   ID               35 non-null    int64
1   Experience_Years 35 non-null    int64
2   Age              35 non-null    int64
3   Gender           35 non-null    object
4   Salary           35 non-null    int64
dtypes: int64(4), object(1)
memory usage: 1.5+ KB
```